

Week 1 Learning Report

1. Why ViT?

For training models over a dataset of images we initially had only the method of CNNs. Under CNN we majorly focused on smaller blocks or patches of images and the overall relation came very late in layers. This required a lot of computation power and was inefficient. Therefore taking motivation from the NLP-related transformers ViT was developed -> it helped maintaining a correlation amongst different patches from the very first layer itself.

DATA SPECIFIC REASONS : Galaxy shape is about large-scale structure such as spiral arms, bars and tidal tails. In a ViT every part of the image can interact with every other part from the very first layer. ViTs are also the backbone of self-supervised models like DINOv2, which learn useful features without labels. That's valuable because expert galaxy labels are limited and noisy.

2. How image patchification works

A Transformer expects a sequence of tokens, so the first step is to turn the image into one. The image is cut into small, non-overlapping squares called patches.

- Number of patches = (image height ÷ patch size) × (image width ÷ patch size).
- The original ViT uses 16×16 patches: a 224×224 image gives $14 \times 14 = 196$ patches.
- **DINOv2 uses 14×14 patches:** a 224×224 image gives **$16 \times 16 = 256$ patches**.
- Each patch is flattened into a list of numbers: $14 \times 14 \text{ pixels} \times 3 \text{ colours} = 588$ values.

The image size must be a multiple of the patch size. Our DESI galaxy images are 160×160, and $160 \div 14 = 11.4$ isn't a whole number, so we resize them to 224×224 before giving them to DINOv2.

3. Patch embeddings

Each flattened patch (588 numbers) is passed through one linear layer that turns it into a vector of **768 numbers**. This vector is called the patch embedding, or token.

- The **same** layer is used for every patch, so all patches are described in the same language.
- 768 is the model's width . It stays the same through the whole network.
- At this stage a token only describes the pixels of its own square. The context comes later, in the Transformer layers.

You can think of it like turning each word of a sentence into a word vector before a language model reads it.

4. Positional embeddings

Self-attention on its own has no sense of order: if you shuffled the patches, it wouldn't notice. But position obviously matters in an image, since a patch in the centre of a galaxy is different from one at the edge.

So ViT adds a position embeddings to every token: a learned vector of 768 numbers, one for each position in the grid (plus one for the CLS token). It's simply added to the patch embedding, so each token now carries both "what it looks like" and "where it is".

ViT learns these as simple 1D embeddings, and the model works out the 2D layout on its own (nearby positions end up with similar embeddings).

5. CLS token

The CLS token is one extra learnable vector placed at the front of the patch sequence, so 256 patches becomes 257 tokens.

- It does not come from any part of the image. It starts as the same learned vector for every image.
- In each of the 12 layers it looks at all the patches through attention and gathers information from them.
- By the end it holds a summary of the whole image. It's used as the image's overall description, for example to classify it or find similar images.

There is only one CLS token per image.

6. ViT Transformer architecture

ViT is an encoder-only Transformer: the same block repeated 12 times (base model). Each block does two things:

1. **Multi-head self-attention:** every token compares itself with every other token and mixes in information from the ones that matter. "Multi-head" means this is done 12 times in parallel, so different heads can focus on different kinds of relationships.
2. **MLP (feed-forward network):** a small 2-layer network applied to each token separately.

Each of these has a LayerNorm before it and a residual (skip) connection around it, which keeps training stable. The number of tokens and their size never change, so the dimension of input and output are same.

7. CLS vs patch tokens

CLS token

There is only one CLS token per image, and its shape is (1, 768). It describes the whole image, gathering information from all the patches as it passes through the model's layers. This makes it useful for classification, finding similar images and clustering. Its downside is that it loses the spatial detail of where things are in the image.

Patch tokens

For a 224×224 image there are 256 patch tokens, with shape (1, 256, 768), which can be reshaped into a 16×16 grid of shape (1, 16, 16, 768). Each patch token describes one 14×14 square of the image, along with context it has picked up from the rest of the image. This makes patch tokens useful for finding where things are, as in segmentation or locating specific features. Their downside is that there are many vectors to handle for every image.

For galaxies, the **CLS token** is the natural choice for questions about the whole galaxy, like “smooth or featured?” or “which galaxies look alike?”. The **patch tokens** keep the position information needed to find where things are, such as a spiral arm, a bar or a nearby companion.

8. What DINOv2 is

DINOv2 is a family of Vision Transformers trained with self-supervised learning: no labels and no captions, just dataset of images. The goal was general-purpose image features that work well out of the box.

How it learned without labels:

- **Student and teacher:** two copies of the network. The teacher is a slowly updated average of the student.
- **Image level (DINO):** the student sees different crops of an image and must give the same output as the teacher. It learns that different views of the same thing should match.
- **Patch level (iBOT):** some patches are hidden from the student, and it must predict what the teacher sees there. It learns good local features.

It comes in four sizes: small, base, large and giant (widths 384, 768, 1024 and 1536), all with patch size 14. We use dinov2-base: 12 layers, 12 heads, 768 dimensions, 86.6 million parameters. We use it frozen, which means we never change its weights.

9. What DINOv2 outputs

For one 224×224 image, DINOv2-base returns:

`last_hidden_state`

Shape (1, 257, 768). This is the main output of the model after the final layer. It holds 257 tokens for the image, each described by 768 numbers: 1 CLS token followed by 256 patch tokens.

CLS token

Shape (1, 768). This is the first token of `last_hidden_state`, taken with `last_hidden_state[:, 0, :]`. It is the summary of the whole image.

Patch tokens

Shape (1, 256, 768). These are the remaining 256 tokens, taken with `last_hidden_state[:, 1:, :]`. There is one token for each 14×14 square of the image.

pooler_output

Shape (1, 768). This is the CLS token again, given as a separate output for convenience.

The numbers themselves don't mean anything to a human, but similar images give similar vectors. We compare them with cosine similarity (1 = pointing the same way). Patch tokens are in reading order, so token p sits at row $p \div 16$, column $p \bmod 16$, and they can be reshaped into a 16×16 grid and visualised.

10. How DINOv2 will be used in this project

The project uses **DESI Legacy Survey DR10** galaxy images (160×160 pixels, 4 filters g, r, i, z) and **Galaxy Zoo DESI** labels (volunteer votes on galaxy shape). DINOv2 is used as a frozen feature extractor:

3. **Prepare:** remove sky noise, stretch the brightness, combine z, r, g into a colour image (DINOv2 needs 3 channels), resize to 224×224 and normalise.
4. **Extract:** run DINOv2 and keep the CLS token and the 256 patch tokens for each galaxy.
5. **Explore:** check whether similar galaxies get similar embeddings (PCA plots, nearest-neighbour search).
6. **Predict:** train small, simple models on top of the frozen embeddings to predict Galaxy Zoo labels.
7. **Locate:** use patch tokens to see where features like arms, bars or companions are.

What we found in Week 1:

- On Galaxy Zoo images, the CLS tokens already separate smooth from featured galaxies in a simple PCA plot, without any training.
- Nearest-neighbour search returns spirals for a spiral and round blobs for a round blob.
- A simple classifier on the CLS tokens reached 84.7% accuracy (smooth vs featured) vs 74% for always guessing “smooth”.
- On our DESI cutouts, the patches most similar to the CLS token sit on the bright galaxies, not the empty sky.